Progree DevOps Internship - Task 4

Haseeb Ullah
Automated Infrastructure as Code and Orchestration Architecture

Selected execution environment: local Minikube on Windows with Docker Desktop. This report separates observed Kubernetes results from an undeployed cloud network design.

PDF requirement	Implementation and evidence
Containerized stack on Kubernetes	Wanderlust frontend/backend, MongoDB and Redis, standalone Task 4 namespace.
Persistent volume claims	1 GiB MongoDB PVC; stored record survived database pod replacement.
Ingress controller endpoints	Traefik Ingress with actual browser/API traffic through localhost:8084.
Horizontal Pod Autoscaler	Backend CPU HPA; actual expansion and cooldown recorded.
Declarative isolated cloud networks	Terraform AWS VPC/subnet/route blueprint; validation and mocked tests passed. No cloud apply.
Elastic, highly available cluster	Local pod elasticity and recovery tested. Single-node laptop and singleton database are not fully HA.

Completion boundary: local orchestration demonstrated; live cloud networking and node/control-plane/database high availability remain unverified. This is not a claim that the full cloud/HA objective has been completed.

Separate Task 4 GitHub repository

Verified cluster and application

Check	Observed result
Cluster	progree-task4
Kubernetes	v1.37.0
Cluster verification time	2026-10-07T10:24:49.578221+00:00
Node Ready	True
Metrics API	True
Ingress controller	Traefik Ready
Default StorageClass	standard
Application checks	passed
Application check time	2026-10-07T10:25:06.790262+00:00

PASS: Frontend and direct SPA navigation respond

PASS: Backend, MongoDB, and Redis are ready through the Ingress

PASS: Invalid post is rejected

PASS: Email/password API signup and signin use the runtime JWT secret

PASS: Create, read, and list a post backed by MongoDB

PASS: Update is visible in both detail and cached list routes

PASS: Delete removes the post from MongoDB and cached lists

Routing: Windows localhost port-forward -> Traefik -> frontend or backend Service -> ready pods. Database and cache use internal ClusterIP services. No cloud load balancer or permanent public endpoint was created.

Persistence, recovery and elasticity

Check	Observed result
MongoDB pod replacement	passed
Same PVC retained	True
Persistent volume	pvc-3c4696b5-82a1-4382-88f9-4fb10d3017ca
Backend replacement	passed
HPA scale-up	passed
HPA maximum observed replicas	4
HPA final replicas	2
HPA cooldown	passed

The persistence test creates one temporary post, replaces the database pod, verifies the same record using the same PVC, and removes only its test record. The backend test replaces one pod and checks that ready replicas and API access recover.

The load test sends concurrent HTTP requests to the existing backend health endpoint. It observes the HPA rather than manually changing replica counts, removes the load job, and waits for the replica count to settle to two.

These tests do not prove zero downtime or tolerance of laptop, worker-node, control-plane, disk, or database-instance failure. Pod replicas share one node; the database has persistent storage but no replica set.

Infrastructure code and presentation

Terraform check	Result
Version	1.16.5
AWS provider	hashicorp/aws 6.67.0
Configuration validation	passed
Mock tests passed	2
Cloud resources created	False

The optional AWS module declares an isolated VPC, four subnets across two availability zones, an internet gateway and separate public/private route tables. Private subnets have no internet route or NAT. Mocked plans require no credentials and do not prove that AWS resources exist. This module does not create EKS or connect Minikube to AWS.

Presentation: explain the desired-state manifests; show Ready pods, the running application, the bound PVC, replacement evidence and HPA samples; then explain the Terraform network and its explicit cloud-deployment boundary.

Reproduce: scripts/start-cluster.ps1; py -3 scripts/deploy.py; scripts/serve.ps1; py -3 scripts/verify-stack.py; py -3 scripts/verify-resilience.py; py -3 scripts/verify-hpa.py. Detailed instructions are in the repository README.

Security and attribution: credentials are generated locally and excluded from Git. A namespace is not network isolation; the default kindnet CNI does not enforce NetworkPolicy. Kubernetes Secrets require additional protection for production. Wanderlust is retained under its MIT license. Its inherited post API lacks authorization and login screens are not fully integrated; use this as a local internship demonstration.

Remaining cloud/HA work: authorize a cloud provider/account/region and budget, validate a real plan, deploy the network and a multi-zone cluster, add replicated storage/database and backups, and verify failure tolerance. No cloud resources or internship submission were created by this report.